TR-032 · FOLIO 01 · READING III · 2026-09-15

Third Reading

An independent specification reading of LLRLANG 0.3.2. Interpreter written from the language reference alone. No shared code with the C seed, the self-hosted compiler, or the project oracle.

CONTRADICTION	GAPS	RUNNABLE	REJECTED
1	9	53	46

ORACLE · FINAL PASS · 53 / 46 / 12 / 4

SECTION 1

What this is

GPT and Claude already read 0.3.1 from the reference. You closed what they found. This is the same job against 0.3.2: an interpreter derived from the text, a new corpus, and the places two honest implementations can still diverge.

The headline is not "the compiler is wrong." It is that a local can be in scope without ever having been assigned; that `main`'s parameter names are required and conventional at once; and that string identity, `return` unwinding, and `argv` layout are still silent.

SECTION 2

Contradiction

CONTRADICTION · §1 vs §9

main parameter names

> *Section 1: "Conventionally they are named argc and argv." Section 9: "main must accept exactly (argc argv)."*

"Conventionally" means the names are a convention. The diagnostic means they are the contract. This reading enforces the names, because a diagnostic table a third implementation is asked to match is the stronger sentence.

```
(fn main (a b)
  (return 0))
```

Ask. Delete "conventionally", or drop the name check and keep arity-2. Add `tests/negative/main_wrong_names.llr` once you pick.

SECTION 3

Gaps

GAP · HEADLINE · §3

Uninitialized locals after a let that never ran

A `let` is a static declaration plus a dynamic assignment. Source-order scope means `x` is visible after an untaken branch. The reference never says what `x` contains.

```
(fn main (argc argv)
  (if 0 (block (let x 1)))
  (return x))
```

A zero-filling interpreter, a poison interpreter, and a native 2,048-byte frame of stack garbage are all licensed. This oracle zero-fills. Probe exits 0.

Ask. Specify that every local slot is 0 until its `let` executes — the only option that keeps native and oracle comparable.

GAP · §3

Is a let initializer in scope of the name?

```
(let x x)
```

“From the point of its let” can mean the name token or the completed form. This reading allows it and yields 0. Recommend: the initializer is not in scope.

GAP · §2

String-literal identity

```
(== "a" "a")
```

Same address or not? Unspecified. This oracle does not intern (exits 0). A coalescing native image will return 1, and the 0.3.0 oracle gate can fail on a program the spec authorises both to accept.

GAP · §4

Empty form ()

The example of a non-identifier head is `(1 2)`. `()` has no first element. This reading rejects it as `expected form name`.

GAP · §4 vs §5–7

Operand-count exactness is only claimed for §5–7

`(let x 1 2)`, `(while 1 (block) 0)`, `(return 1 2)` are not covered. This reading treats §4 forms as exact too. Move the sentence, or table the arities.

GAP · §4, §10

return unwinding

```
(return (+ (return 1) (/ 1 0)))
```

This reading exits 1 and does not trap. Write that `return` is a non-local exit of the current function that skips remaining operands.

GAP · §8

argv wording

“Points to argc NUL-terminated strings followed by a null pointer” is `char*` concatenated, or `char**`? This reading lays out a Linux pointer table. Ask: “argv is a pointer to argc pointers to NUL-terminated strings, followed by a null pointer.”

GAP · §2

SIGSEGV numeric status

The reference says SIGSEGV. It never says 139. Changelog fixtures use 139. Python `subprocess` reports −11. Treat death-by-signal as matching regardless of wait encoding.

GAP · §3–5

Three namespaces, never named

```
(fn + (a b) (return 99))
(fn main (argc argv)
  (return (+ (call + 1 2) 1)))   ; 100
```

Locals, functions, and builtin forms are different lookups. No reserved-word list. One paragraph under §3.

SECTION 4

Suggested 0.3.3

1. Resolve §1 vs §9 on `main` names.
2. Specify local-slot initial value (recommend 0).
3. Initializer of `let` is not in scope of the name.
4. String-literal identity is not to be depended on.
5. `return` is a non-local function exit.
6. `argv` is `char**`.
7. Exact operand counts cover §4; name `()`.
8. Name the three namespaces.

None of these require a code-generation change if the native compiler already zero-fills, does not intern, and already rejects `(fn main (a b) …)`. If it does not zero-fill, (2) is a real native/oracle split and should be gated.

SECTION 5

Oracle choices

TOPIC	THIS READING
Uninitialized local	0
`(let x x)`	in scope, yields 0
Equal string literals	distinct addresses

TOPIC	THIS READING
Empty `(block)`	0 (already unspecified)
`main` names	must be `argc argv`
`()`	`expected form name`
§4 extra operands	rejected
Guest syscalls	write, mmap, munmap, exit; else –ENOSYS
Call depth	8192 then SIGSEGV
Diagnostic prefix	`llri:` (text not required to match `llrc`)

SECTION 6

Corpus

BUCKET	N	JUDGED
positive	53	exit + stdout
negative	46	exit 2, diagnostic substring
runtime	12	120 / 121 / 122 / 123 / 139
gap-probes	4	recorded, not judged

Run with `python3 scripts/run_review.py`. Captured evidence: `results/RUN.txt`. Last line: FINAL PASS.

0.3.2 already closed quote-as-delimiter, use-before-let, parameter slots, nesting from depth 1, store-to-string SIGSEGV, % traps like `/`, else-less `if`, form-head identifier, LF-only line starts. This reading agrees.